출력층 설계: 항등 함수 vs 소프트맥스 함수

신경망은 회귀·분류 모두에 쓸 수 있고, 출력층의 활성화 함수만 달라진다

회귀 (regression)

항등 함수 identity

입력을 그대로 출력한다

$y = a$

분류 (classification)

소프트맥스 함수 softmax

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_i)}$$

분자: 입력 신호의 지수함수 분모: 모든 입력 지수함수의 합

개념 이해 예시: a = [0.3, 2.9, 4.0]

exp(a)

`[ 1.35,  18.17,  54.60 ]`

→

합계

`74.12`

→

y = exp(a) / 합계

`[ 0.018,  0.245,  0.737 ]`

소프트맥스 구현 시 주의점: 오버플로우

지수 함수는 값이 매우 빨리 커져서, 입력이 크면 계산이 깨진다

문제

a = [1010, 1000, 990]

exp(1010) 같은 값은 표현 범위를 넘어 inf가 되고,
inf / inf = nan → 제대로 계산이 안 됨

```
np.exp(a) / np.sum(np.exp(a))
# overflow -> nan, not a valid result
```

개선: 분모·분자에 같은 상수를 더해 준다

$$y_k = \frac{\exp(a_k)}{\sum_i \exp(a_i)} = \frac{\exp(a_k + C')}{\sum_i \exp(a_i + C')}$$

C′ 값은 무엇이든 결과가 같다.
일반적으로 입력 신호 중 최댓값을 사용한다.

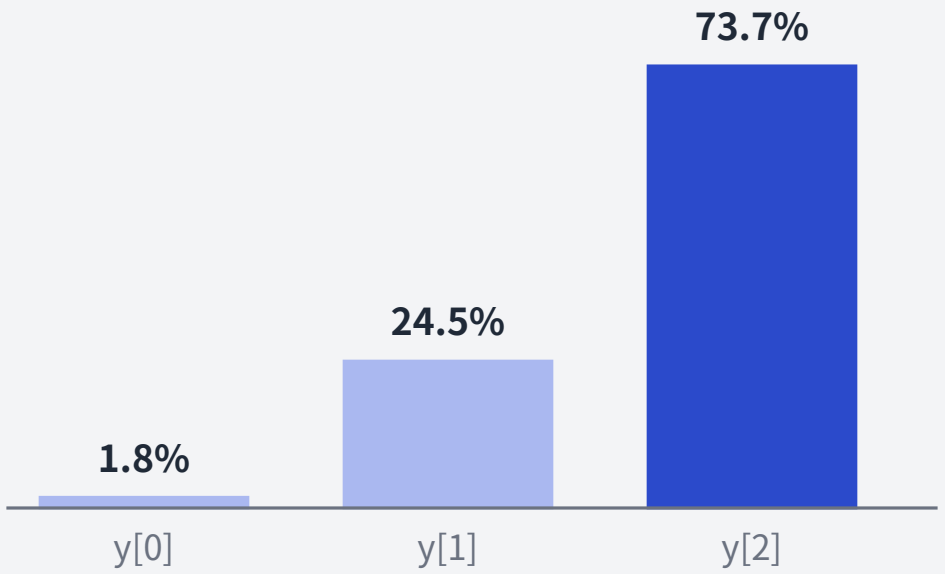
```
def softmax(a):
    c = np.max(a)
    exp_a = np.exp(a - c)   # overflow guard
    return exp_a / np.sum(exp_a)
```

소프트맥스의 특징과 출력층 뉴런 수

출력을 "확률"로 해석할 수 있어서 분류 문제에 적합하다

a = [0.3, 2.9, 4.0] → softmax(a)

가장 큰 값 y[2] → 인덱스 2번 클래스로 판단



소프트맥스의 특징

- 출력은 0과 1 사이의 값
- 출력의 총합은 1 → 확률로 해석 가능
- 추론 시 출력층의 소프트맥스는 생략 가능
  (exp는 단조 증가라 가장 큰 값의 위치가 그대로)

출력층의 뉴런 수 = 분류하려는 클래스 수

숫자 0~9 분류 → 출력 뉴런 10개

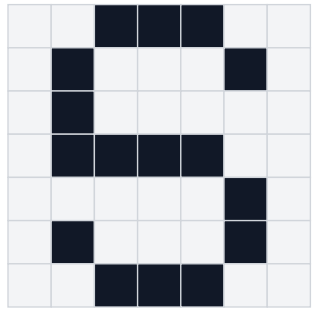
0 1 2 3 4 5 6 7 8 9

MNIST 데이터셋

학습은 생략하고, 이미 학습된 매개변수로 추론(순전파)만 구현한다

60,000	10,000	28 × 28	0 ~ 255
훈련 이미지	시험 이미지	회색조 이미지	픽셀 값

각 이미지에는 그 이미지가 나타내는 숫자(0~9)의 라벨이 붙어 있다



28 × 28 이미지 (개념도)



784개 (1차원 배열)

load_mnist() 인수

`flatten`	이미지를 1차원 배열로 만들지 여부
`normalize`	픽셀 값을 0~1로 정규화할지 여부
`one_hot_label`	라벨을 원-핫 인코딩으로 저장할지 여부

원-핫 인코딩: 정답은 1, 나머지는 0인 배열

신경망의 추론 처리

입력 784 → 은닉층 50 → 은닉층 100 → 출력 10



구성 함수

`init_network()`
학습된 가중치(pkl) 읽기

`get_data()`
MNIST 시험 데이터 읽기

`predict(network, x)`
신경망 순전파

```
a1 = np.dot(x, W1) + b1;   z1 = sigmoid(a1)
a2 = np.dot(z1, W2) + b2;  z2 = sigmoid(a2)
a3 = np.dot(z2, W3) + b3;  y  = softmax(a3)
p = np.argmax(y)   # index of the highest probability
```

정확도 계산

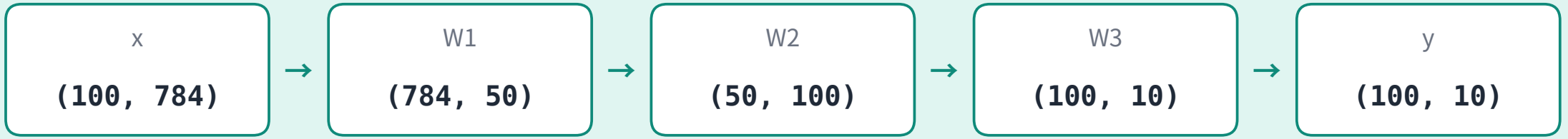
예측 p 와 정답 t[i]가 같으면 카운트
→ 맞힌 수 / 전체 수 = Accuracy

정규화: 데이터를 특정 범위로 변환 (전처리의 한 예)

배치 처리 (batch)

이미지 여러 장을 하나로 묶어 한 번에 넘기면 행렬 연산으로 처리된다

배열 형상 (batch_size = 100)




```
for i in range(0, len(x), batch_size):
    x_batch = x[i:i+batch_size]
    y_batch = predict(network, x_batch)
    p = np.argmax(y_batch, axis=1)
    accuracy_cnt += np.sum(p == t[i:i+batch_size])
```

`np.argmax(x, axis=1)`

각 행(이미지)마다 최댓값의 인덱스를 찾는다

`[[.1,.8,.1],[.3,.1,.6],[.2,.5,.3],[.8,.1,.1]]`

→ **`[1, 2, 1, 0]`**